

Natural Language Processing (AI408DV01)

Bài tập thực hành cá nhân – Tuần 07

Name:	Nguyễn Phúc Minh Châu
Student ID:	22302421
Deadline:	04/05/2026 - 11/05/2026
File name format:	AI301DV01-Lab-07-22302421.pdf

A. Chuẩn bị

- Đọc kỹ file/slide bài giảng **Dependency Parsing**

B. Câu hỏi lý thuyết

Bài tập 1.

Hãy nêu sự khác biệt cơ bản giữa phân tích ngữ pháp phụ thuộc (dependency parsing) và phân tích ngữ pháp dựa trên thành phần (constituent-based parsing).

Đặc Điểm	Constituent-based Parsing	Dependency Parsing
Lý Thuyết	Dựa trên ngữ pháp phi ngữ cảnh (CFG) và cấu trúc cụm từ	Dựa trên lý thuyết cú pháp phụ thuộc, mô tả quan hệ nhị phân giữa các từ
Cấu Tạo	Chia câu thành các khối cấu trúc phân cấp lồng nhau (như NP, VP)	Liên kết trực tiếp các thành phần từ vựng bằng các mối quan hệ bất đối xứng
Tree Structure	Các node trung gian là các ký hiệu trừu tượng (non-terminals) như cụm danh từ, cụm động từ	Không có node cụm từ trung gian; mọi node trong cây đều là từ root
Relations	Thể hiện cách các từ nhóm lại thành một đơn vị lớn hơn	Thể hiện quan hệ ngữ pháp cụ thể (như chủ ngữ, tân ngữ) giữa từ chính (head) và từ phụ thuộc (dependent)

Ưu Điểm	Tốt cho việc xác định cấu trúc phân cấp và ngữ pháp chính quy của ngôn ngữ có trật tự từ cố định	Xử lý tốt các ngôn ngữ có trật tự từ tự do (free word order) và gần gũi với ý nghĩa ngữ nghĩa của câu
---------	--	---

Tại sao phân tích ngữ pháp phụ thuộc lại được coi là có lợi thế hơn đối với các ngôn ngữ có trật tự từ tự do (free word order).

Phân tích ngữ pháp phụ thuộc có lợi thế rõ rệt với các ngôn ngữ trật tự từ tự do vì nó biểu diễn quan hệ ngữ pháp bằng một liên kết duy nhất giữa từ chính và từ phụ thuộc, thay vì cần nhiều quy tắc cấu trúc cụm từ riêng biệt cho từng vị trí khác nhau của từ trong câu.

Cách tiếp cận này trừu tượng hóa hoàn toàn khỏi trật tự tuyến tính, tập trung vào chức năng ngữ pháp — điều đặc biệt quan trọng khi chức năng đó được mã hóa qua hình thái học thay vì vị trí. Hơn nữa, cấu trúc phụ thuộc liên kết trực tiếp động từ với các tham số của nó, giúp việc trích xuất quan hệ ngữ nghĩa "ai làm gì cho ai" trở nên ổn định và hiệu quả bất kể từ được sắp xếp theo thứ tự nào.

Bài tập 2.

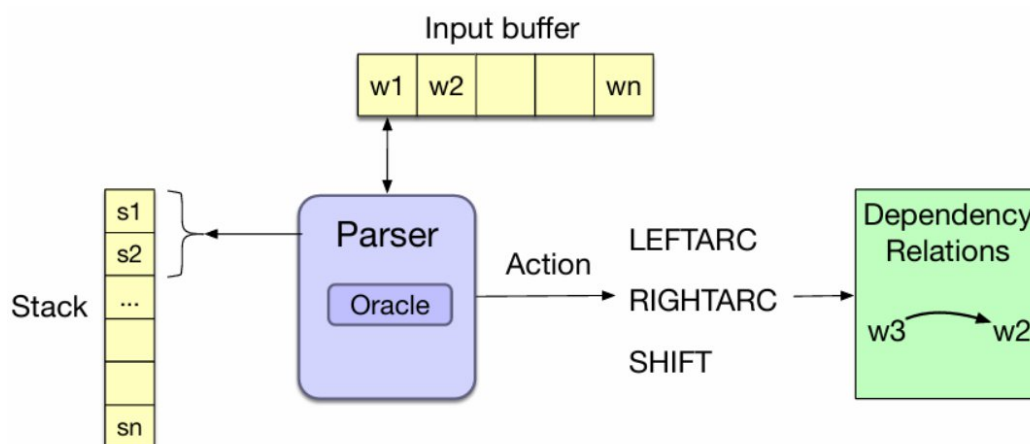
Trình bày cơ chế Phân tích dựa trên Chuyển đổi (Transition-Based Parsing)?

Cơ chế Transition-Based Parsing là một phương pháp xây dựng cây phụ thuộc cho câu văn dựa trên mô hình shift-reduce. Một bộ phân tích cú pháp dựa trên chuyển tiếp bao gồm ba thành phần cốt lõi để quản lý trạng thái (cấu hình) của quá trình phân tích:

- **Ngăn xếp (Stack - σ):** Dùng để lưu trữ các từ đang được xử lý, bắt đầu với **ROOT**.
- **Bộ đệm đầu vào (Input Buffer - β):** Chứa danh sách các từ trong câu chưa được xử lý theo thứ tự từ trái sang phải.
- **Tập hợp các quan hệ phụ thuộc (A):** Lưu trữ các cung (arcs) có nhãn đại diện cho quan hệ ngữ pháp giữa từ chính (head) và từ phụ thuộc (dependent) đang được xây dựng dần dần.

- **Bộ dự đoán (Oracle/Predictor):** Đóng vai trò quyết định hành động chuyển trạng thái tiếp theo dựa trên cấu hình hiện tại của ngăn xếp và bộ đệm

Trong **Arc-Standard** hệ thống thực hiện ba loại toán tử chính là **Shift** - lấy từ ở đầu bộ đệm đầu vào và đẩy (push) vào ngăn xếp; **Left-Arc** - tạo một quan hệ phụ thuộc $w_{top} \rightarrow w_{below_top}$ (trong đó từ ở đỉnh ngăn xếp là từ chính, từ ngay dưới nó là từ phụ thuộc), sau đó loại bỏ từ phụ thuộc khỏi ngăn xếp; **Right-Arc** - Tạo một quan hệ phụ thuộc $w_{below_top} \rightarrow w_{top}$ (từ dưới đỉnh ngăn xếp làm từ chính, từ ở đỉnh làm từ phụ thuộc), sau đó loại bỏ từ phụ thuộc ở đỉnh khỏi ngăn xếp.



Transition-Based Parsing vận hành như một vòng lặp trạng thái, bắt đầu từ cấu hình khởi tạo với Stack chứa **ROOT**, Buffer chứa toàn bộ câu và tập quan hệ rỗng. Tại mỗi vòng lặp, một bộ phân loại — thường được gọi là Oracle trong bối cảnh huấn luyện có giám sát — quan sát cấu hình hiện tại và chọn hành động chuyển đổi tối ưu (SHIFT, LEFT-ARC hoặc RIGHT-ARC), từ đó tạo ra cấu hình trạng thái mới. Quá trình này lặp lại liên tục và kết thúc khi Buffer rỗng và Stack chỉ còn nút **ROOT**, lúc đó toàn bộ các cung được tích lũy trong tập A tạo thành cây phụ thuộc hoàn chỉnh cho câu đầu vào.

Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$

1. Shift $\sigma, w_i | \beta, A \rightarrow \sigma | w_i, \beta, A$

2. Left-Arc_r $\sigma | w_i, w_j | \beta, A \rightarrow \sigma, w_j | \beta, A \cup \{r(w_j, w_i)\}$

3. Right-Arc_r $\sigma | w_i, w_j | \beta, A \rightarrow \sigma, w_i | \beta, A \cup \{r(w_i, w_j)\}$

Finish: $\beta = \emptyset$

Bài tập 3.

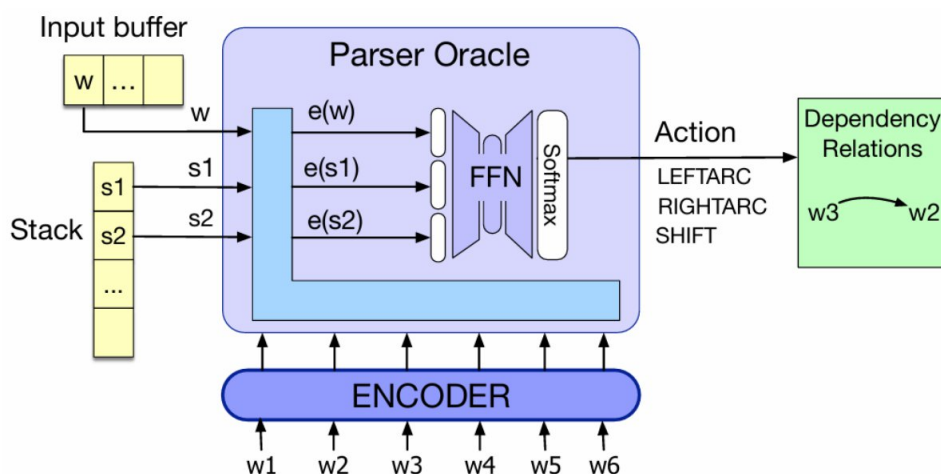
Trình bày cách xây dựng bộ dự đoán (predictor/oracle) trong Phân tích cây cú pháp phụ thuộc (Dependency parsing).

Bộ dự đoán (**Oracle**) trong **transition-based dependency parsing** là một **bộ phân loại** nhận cấu hình trạng thái (**configuration**) hiện tại — gồm Stack, Buffer và tập quan hệ đã xác lập. Bộ dự đoán được training qua học máy có giám sát (**supervised machine learning**) cùng toán tử chuyển trạng thái tương ứng (**SHIFT, LEFT-ARC, RIGHT-ARC**) để dự đoán hành động chuyển tiếp (**Predicted Action**) nhằm dần hình thành cây phụ thuộc.

- Phát sinh các cấu hình (**configuration**) huấn luyện

Step	Stack	Word List	Predicted Action
0	[root]	[book, the, flight, through, houston]	SHIFT
1	[root, book]	[the, flight, through, houston]	SHIFT
2	[root, book, the]	[flight, through, houston]	SHIFT
3	[root, book, the, flight]	[through, houston]	LEFTARC
4	[root, book, flight]	[through, houston]	SHIFT
5	[root, book, flight, through]	[houston]	SHIFT
6	[root, book, flight, through, houston]	[]	LEFTARC
7	[root, book, flight, houston]	[]	RIGHTARC
8	[root, book, flight]	[]	RIGHTARC
9	[root, book]	[]	RIGHTARC
10	[root]	[]	Done

Dữ liệu huấn luyện được tổng hợp bằng cách giả lập quá trình phân tích trên các cây chuẩn (**gold tree**) từ treebank, chuyển đổi chúng thành chuỗi cặp **[configuration, action]** theo các quy tắc xác định. Bộ phân loại có thể được xây dựng theo hướng **feature-based** (dùng template đặc trưng thủ công kết hợp SVM/Logistic Regression) hoặc **neural** (dùng encoder BiLSTM/BERT để trích xuất vector ngữ cảnh rồi phân loại qua FFN + Softmax). Hiệu quả của Oracle được đánh giá qua UAS và LAS, đồng thời vấn đề lỗi tích lũy do quyết định tham lam (Greedy) được giảm thiểu bằng **Beam Search** hoặc **Dynamic Oracle**.



Bài tập 4.

Trình bày các tiếp cận Phân tích cú pháp phụ thuộc dựa trên đồ thị (Graph-Based Dependency Parsing).

Graph-Based Dependency Parsing không thực hiện các quyết định cục bộ mà xem xét và chấm điểm trên toàn bộ cấu trúc cây cú pháp. Graph-Based Dependency Parsing sử dụng tính điểm dựa trên cạnh (Edge-Factored) để tìm kiếm trong không gian tất cả các cây cú pháp để tìm ra một hoặc vài cây tối ưu hóa điểm số (Score). Lấy điểm số của một cây (t) cho câu (S) được giả định bằng tổng điểm số của tất cả các cạnh (e) cấu thành nên cây đó.

$$Score(t, S) = \sum_{e \in t} Score(e)$$

Thuật toán Cây khung cực đại (Maximum Spanning Tree - MST) khi bài toán phân tích cú pháp lúc này trở thành việc tìm một cây khung (spanning tree) có tổng điểm lớn nhất trong một đồ thị có hướng, đầy đủ, nơi các đỉnh là các từ trong câu và các cạnh là các quan hệ head-dependent tiềm năng.

Trong đó thuật toán Chu-Liu/Edmonds (CLE) là thuật toán phổ biến nhất để tìm MST trong môi trường đồ thị có hướng, thuật toán bắt đầu bằng cách chọn cạnh đi vào có điểm cao nhất cho mỗi nút; nếu tạo thành chu trình, nó sẽ "co" chu trình đó lại thành một siêu nút và điều chỉnh điểm số các cạnh liên quan cho đến khi thu được một cây khung hợp lệ không có chu trình.

- Đồ thị có hướng (directed graph), có gốc ban đầu cho câu “Book that flight”

